UNIVERSITAS
DIAN NUSANTARA

# Struktur Data dan Algoritma

## Linked List

Fakultas : FTI

Program Studi : TEKNIK INFORMATIKA

Tatap Muka

03

Kode Matakuliah : 412523301

Disusun oleh : Riad Sahara, S.SI., M.T.

# DESKRIPSI MATERI PERTEMUAN 3

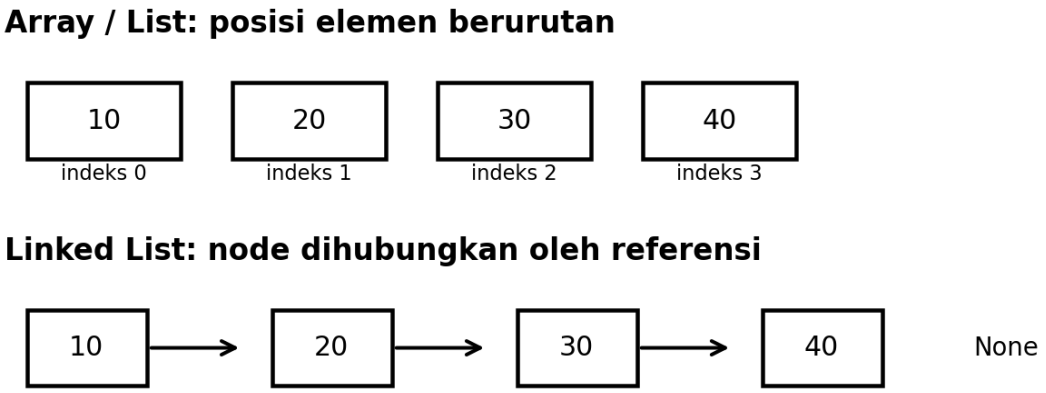
| | |
|---|---|
| Materi Pertemuan 3 | Linked List |
| Deskripsi Materi | Materi membahas konsep linked list, node dan referensi, Singly Linked List, operasi traversal, pencarian, penambahan, pembaruan, dan penghapusan node, analisis kompleksitas, pengantar Doubly Linked List dan Circular Linked List, serta implementasi langkah demi langkah menggunakan Python di Google Colab. |
| Sub Capaian Pembelajaran Mata Kuliah (Sub CPMK) | Luaran yang diharapkan setelah mengikuti materi pada pertemuan ini:<br>• Mahasiswa mampu menjelaskan konsep node, head, data, next, dan hubungan antar-node.<br>• Mahasiswa mampu membedakan karakteristik array/list dengan linked list.<br>• Mahasiswa mampu membangun Singly Linked List menggunakan class Python.<br>• Mahasiswa mampu mengimplementasikan traversal, search, insert, update, dan delete.<br>• Mahasiswa mampu melakukan dry run dan menjelaskan perubahan referensi pada linked list.<br>• Mahasiswa mampu menganalisis kompleksitas operasi dasar linked list dan memilihnya pada kasus yang tepat. |
| Deskripsi Tugas | Latihan formatif dan praktik Google Colab meliputi pembuatan class Node dan SinglyLinkedList, penelusuran node, operasi insert/delete, pencarian data, pengujian beberapa edge case, serta mini studi kasus daftar tugas. |
| Kontrak Perkuliahan | Ketentuan perkuliahan:<br>1. Mahasiswa menyiapkan notebook Google Colab Pertemuan 3 dan menjalankan seluruh contoh kode sendiri.<br>2. Mahasiswa wajib memahami perubahan referensi, tidak hanya menyalin kode.<br>3. Setiap praktik disertai output, dry run singkat, dan penjelasan kompleksitas operasi.<br>4. Komposisi penilaian mengikuti RPS resmi dan ketentuan kelas yang berlaku. |

# Pertemuan 3
# Linked List

## 3.1. Pengantar Linked List

Pada pertemuan sebelumnya, mahasiswa telah mempelajari array dan algoritma pencarian. Array sangat baik ketika kita membutuhkan akses berdasarkan indeks. Namun, beberapa operasi seperti menyisipkan atau menghapus elemen di awal dan di tengah dapat memerlukan pergeseran banyak elemen. Linked list menawarkan cara penyimpanan yang berbeda: data disimpan di dalam node, dan setiap node menyimpan referensi menuju node berikutnya.

Linked list penting karena memperkenalkan gagasan bahwa hubungan antar-data tidak harus ditentukan oleh posisi indeks yang berurutan. Struktur ini juga menjadi fondasi konseptual untuk berbagai struktur data lain seperti stack, queue, adjacency list pada graph, serta beberapa teknik pengelolaan memori dan struktur dinamis.




Gambar 3.1 Perbedaan konseptual array/list dan linked list
Sumber: Diolah oleh penulis.

### Tujuan Pembelajaran Pertemuan 3

- Memahami konsep node, head, data, next, dan referensi antar-node.
- Mampu membangun Singly Linked List dari awal menggunakan Python.
- Mampu melakukan traversal, search, insert, update, dan delete.
- Mampu menjelaskan perubahan referensi ketika struktur linked list berubah.
- Mampu membandingkan karakteristik linked list dengan array/list.
- Mampu menganalisis kompleksitas operasi dasar linked list.

Prinsip utama pada pertemuan ini: jangan menghafal kode. Setiap kali satu baris mengubah atribut next, gambarkan terlebih dahulu node mana yang sekarang menunjuk ke node mana. Pemahaman referensi lebih penting daripada panjang kode.

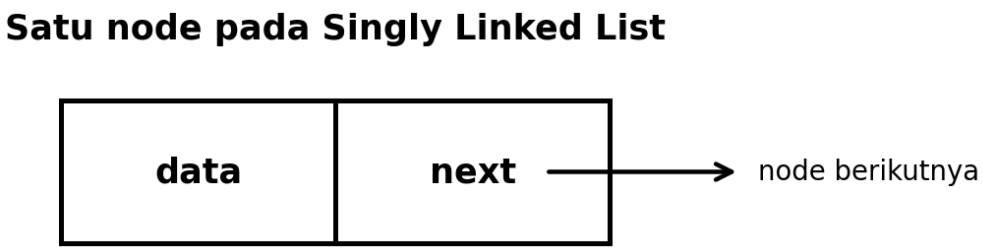
## 3.2. Konsep Dasar Linked List

### 3.2.1 Apa yang Dimaksud dengan Linked List?

Linked list adalah struktur data linear yang tersusun dari sekumpulan node. Setiap node menyimpan data dan satu atau lebih referensi yang menghubungkannya dengan node lain. Pada Singly Linked List, setiap node memiliki satu referensi utama bernama next yang menunjuk ke node berikutnya.

Berbeda dengan array, linked list tidak mengandalkan indeks untuk berpindah langsung menuju elemen tertentu. Untuk mencapai node ketiga, misalnya, program harus mulai dari head lalu mengikuti next dari node pertama ke node kedua dan seterusnya. Karena itu, akses elemen berdasarkan posisi umumnya memerlukan traversal.

### 3.2.2 Anatomi Sebuah Node

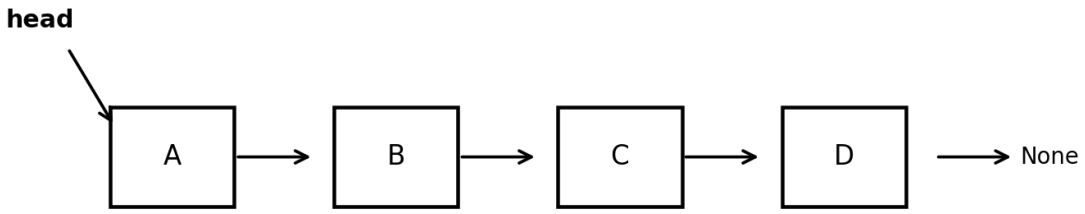



Gambar 3.2 Komponen dasar sebuah node pada Singly Linked List
Sumber: Diolah oleh penulis.

Secara konseptual sebuah node pada Singly Linked List memiliki dua bagian. Bagian data menyimpan nilai yang dibutuhkan aplikasi. Bagian next menyimpan referensi ke node berikutnya. Jika sebuah node merupakan node terakhir, nilai next biasanya None.

| Komponen | Makna | Contoh |
|---|---|---|
| data | Nilai yang disimpan oleh node | NIM, nama, angka, objek |
| next | Referensi menuju node berikutnya | node_b atau None |
| head | Referensi ke node pertama | head → node pertama |
| None | Menandakan tidak ada node berikutnya | next = None pada node terakhir |

### 3.2.3 Head dan Akhir Linked List




Gambar 3.3 Struktur dasar Singly Linked List
Sumber: Diolah oleh penulis.

Head bukan node tambahan. Head adalah referensi yang menunjuk ke node pertama. Jika linked list masih kosong, head bernilai None. Node terakhir tetap merupakan node biasa, tetapi atribut next miliknya bernilai None. Dua kondisi ini akan sering digunakan untuk menentukan apakah list kosong dan kapan traversal harus berhenti.

## 3.3. Perbandingan Array/List dan Linked List

Tidak ada struktur data yang selalu paling baik untuk semua masalah. Array/list dan linked list memiliki kekuatan yang berbeda. Pemilihan struktur data harus mempertimbangkan operasi yang paling sering dilakukan.

| Aspek | Array / Python list | Singly Linked List |
|---|---|---|
| Akses posisi ke-i | Cepat, O(1) | Harus traversal, O(n) |
| Insert di awal | Dapat memerlukan pergeseran, O(n) | O(1) jika head tersedia |
| Delete di awal | Dapat memerlukan pergeseran, O(n) | O(1) |
| Insert di akhir | Umumnya O(1) amortized pada Python list | O(n) jika hanya memiliki head; O(1) jika menyimpan tail |
| Search berdasarkan nilai | O(n) | O(n) |
| Penggunaan memori | Tidak membutuhkan pointer per elemen | Setiap node menyimpan referensi tambahan |
| Cara navigasi | Indeks | Mengikuti referensi next |

Linked list tidak otomatis lebih cepat daripada array. Keuntungan linked list muncul pada operasi tertentu dan bergantung pada referensi yang tersedia. Misalnya insert di awal dapat O(1), tetapi mencari posisi ke-100 tetap membutuhkan traversal dari head.

## 3.4. Persiapan Praktik Python Pertemuan 3

Praktik tetap menggunakan Google Colab. Tidak ada library tambahan yang wajib dipasang karena linked list akan dibangun sendiri menggunakan class Python. Tujuannya agar mahasiswa melihat dengan jelas bagaimana node dan referensi bekerja.

### 3.4.1 Membuat Notebook Praktik

1. Buka Google Colab melalui browser dan masuk menggunakan akun Google.
2. Buat notebook baru dan ubah nama menjadi SDA_Pertemuan_03_Nama_NIM.ipynb.
3. Buat text cell untuk judul: A. Node, B. Singly Linked List, C. Insert, D. Search dan Update, E. Delete, F. Pengujian.
4. Jalankan setiap contoh kode pada code cell yang terpisah agar perubahan struktur lebih mudah diamati.
5. Gunakan Shift + Enter untuk menjalankan cell. Jika muncul error, baca pesan error sebelum mengubah kode.

### 3.4.2 Uji Lingkungan Python

```
print('Praktik Linked List siap')
print('Pertemuan:', 3)
```

```
Praktik Linked List siap
Pertemuan: 3
```

Jika output di atas muncul, runtime Python telah siap. Pada praktik ini kita tidak memerlukan instalasi package. Kita hanya menggunakan konsep class, object, atribut, while, if, dan fungsi/method yang sudah diperkenalkan secara dasar pada pertemuan sebelumnya.

## 3.5. Langkah 1 — Membuat Class Node

Langkah pertama adalah membuat cetak biru node. Kita menggunakan class agar setiap node memiliki struktur yang sama: satu atribut untuk data dan satu atribut untuk referensi next.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

### 3.5.1 Penjelasan Kode

| Bagian Kode | Penjelasan |
|---|---|
| class Node: | Mendefinisikan class bernama Node. |
| def __init__(self, data): | Constructor yang dijalankan ketika object Node dibuat. |
| self.data = data | Menyimpan nilai yang diberikan ke dalam atribut data milik node. |
| self.next = None | Pada saat dibuat, node belum menunjuk ke node lain. |

Kata self menunjuk pada object yang sedang diproses. Dengan demikian, self.data milik node A berbeda dengan self.data milik node B. Atribut next juga melekat pada masing-masing node.

### 3.5.2 Membuat Object Node

```
node_a = Node(10)
node_b = Node(20)

print(node_a.data)
print(node_a.next)
print(node_b.data)
```

```
10
None
20
```

Pada kondisi ini node_a dan node_b sudah ada, tetapi keduanya belum terhubung. Karena atribut next masih None, masing-masing node berdiri sendiri.

### 3.5.3 Menghubungkan Dua Node

```
node_a.next = node_b

print(node_a.data)
print(node_a.next.data)
```

```
10
20
```

Baris node_a.next = node_b mengubah referensi next milik node_a sehingga menunjuk ke object node_b. Karena itu, node_a.next.data menghasilkan 20. Ini adalah ide inti linked list: hubungan antar-node dibentuk melalui referensi, bukan indeks.

## 3.6. Langkah 2 — Membuat SinglyLinkedList

Setelah memahami satu node, kita membutuhkan object yang mengelola keseluruhan linked list. Pada bentuk paling sederhana, SinglyLinkedList hanya perlu menyimpan head.

```
class SinglyLinkedList:
    def __init__(self):
        self.head = None

linked = SinglyLinkedList()
print(linked.head)
```

```
None
```

Nilai head = None berarti linked list belum memiliki node. Setelah node pertama dimasukkan, head harus diubah agar menunjuk ke node tersebut.

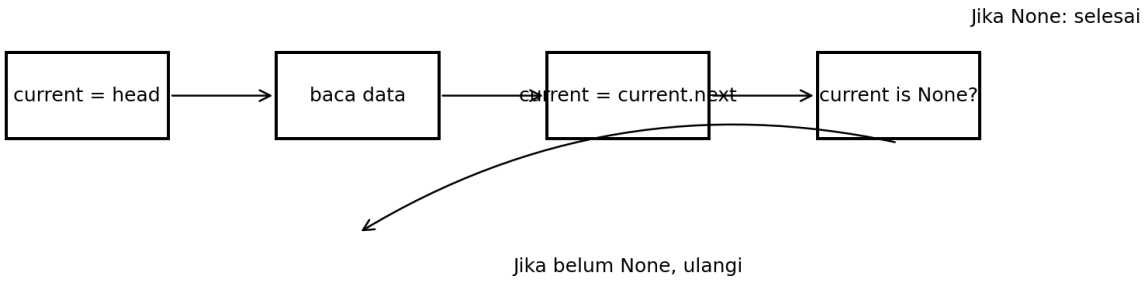
### 3.6.1 Method is_empty()

```
def is_empty(self):
    return self.head is None
```

Ekspresi self.head is None menghasilkan True ketika list kosong dan False ketika sudah memiliki node. Method sederhana ini membantu membuat kondisi program lebih mudah dibaca.

## 3.7. Traversal pada Linked List

Traversal berarti mengunjungi node satu per satu mulai dari head sampai mencapai None. Karena tidak ada indeks yang dapat membawa kita langsung ke node berikutnya, traversal dilakukan dengan variabel bantu yang biasa disebut current.




Gambar 3.4 Alur traversal Singly Linked List
Sumber: Diolah oleh penulis.

### 3.7.1 Algoritma Traversal

```
TRAVERSAL(head)
    current = head
    selama current tidak None:
        proses current.data
        current = current.next
```

Variabel current hanya referensi sementara. Mengubah current = current.next tidak menghapus node. Kita hanya memindahkan posisi pengamatan dari satu node ke node berikutnya.

### 3.7.2 Implementasi display()

```
def display(self):
    current = self.head

    while current is not None:
        print(current.data, end=' -> ')
        current = current.next

    print('None')
```

Kondisi while current is not None memastikan perulangan berjalan selama current masih menunjuk ke node. Pada setiap iterasi, data ditampilkan lalu current dipindahkan ke node berikutnya.

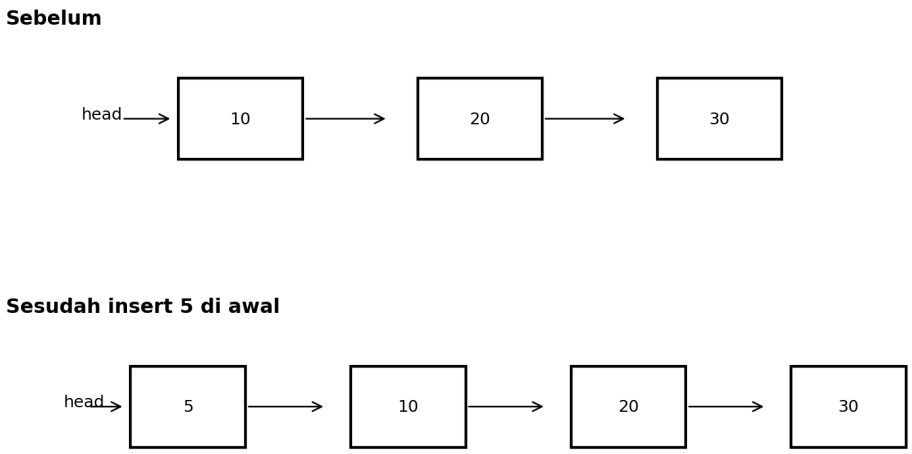
### 3.7.3 Dry Run Traversal

| Iterasi | current.data | current.next | Aksi |
|---|---|---|---|
| 1 | 10 | node 20 | Cetak 10, pindah ke node 20 |
| 2 | 20 | node 30 | Cetak 20, pindah ke node 30 |
| 3 | 30 | None | Cetak 30, lalu current menjadi None |
| Selesai | - | - | Perulangan berhenti |

Karena setiap node dikunjungi tepat satu kali, kompleksitas traversal adalah O(n), dengan n menyatakan jumlah node.

## 3.8. Menambahkan Node di Awal

Insert di awal merupakan salah satu operasi yang menunjukkan keunggulan linked list. Kita tidak perlu menggeser semua elemen. Kita hanya membuat node baru, menghubungkan node baru ke head lama, lalu memindahkan head.




Gambar 3.5 Proses insert node di awal linked list
Sumber: Diolah oleh penulis.

### 3.8.1 Urutan yang Benar

```
INSERT_BEGIN(data)
    new_node = Node(data)
    new_node.next = head
    head = new_node
```

Urutan baris sangat penting. new_node.next harus menunjuk ke head lama sebelum head dipindahkan ke new_node. Jika head dipindahkan terlalu awal tanpa menyimpan hubungan lama, akses ke rangkaian node sebelumnya dapat hilang.

### 3.8.2 Implementasi Python

```
def insert_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
```

Operasi ini tidak bergantung pada jumlah node. Baik linked list berisi 3 node maupun 3 juta node, pekerjaan utamanya tetap membuat node dan mengubah dua referensi. Karena itu kompleksitasnya O(1).

## 3.9. Menambahkan Node di Akhir

Jika class hanya menyimpan head, untuk menambahkan node di akhir kita harus mencari node terakhir terlebih dahulu. Node terakhir dikenali ketika current.next bernilai None.

```
def append(self, data):
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        return

    current = self.head
    while current.next is not None:
        current = current.next

    current.next = new_node
```

### 3.9.1 Mengapa Ada Kasus Khusus List Kosong?

Jika self.head adalah None, tidak ada node yang dapat ditelusuri. Node baru langsung menjadi node pertama. Kata return digunakan agar method berhenti setelah kondisi tersebut selesai dan tidak melanjutkan ke bagian traversal.

### 3.9.2 Kompleksitas Append

Pada implementasi ini append memiliki kompleksitas O(n) karena program harus berjalan dari head hingga node terakhir. Jika class juga menyimpan referensi tail, append dapat dibuat O(1). Kita akan melihat ide ini kembali ketika membahas queue.

## 3.10. Membangun Linked List Pertama Secara Lengkap

Sekarang gabungkan Node, constructor SinglyLinkedList, insert_beginning, append, dan display dalam satu cell agar dapat diuji bersama.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

```
    def append(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node

    def display(self):
        current = self.head
        while current is not None:
            print(current.data, end=' -> ')
            current = current.next
        print('None')
```

### 3.10.1 Uji Program

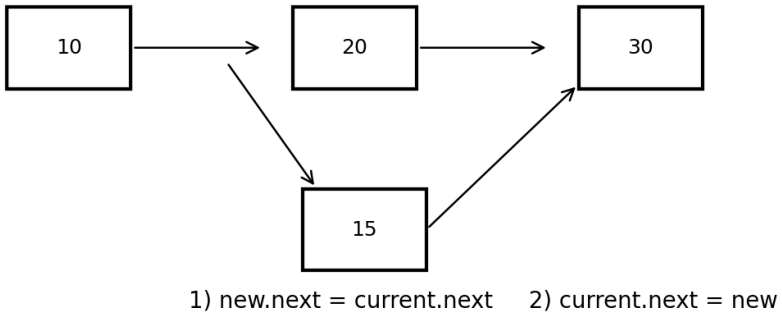
```
linked = SinglyLinkedList()
linked.append(10)
linked.append(20)
linked.append(30)
linked.insert_beginning(5)
linked.display()
```

```
5 -> 10 -> 20 -> 30 -> None
```

Perhatikan bahwa append menambahkan node setelah node terakhir, sedangkan insert_beginning membuat node baru menjadi head. Output harus sesuai urutan operasi yang dilakukan.

## 3.11. Menyisipkan Node Setelah Nilai Tertentu

Kasus berikutnya adalah menambahkan node di tengah. Program perlu mencari node yang menjadi titik penyisipan. Setelah ditemukan, perubahan referensi harus dilakukan dalam urutan yang benar agar sisa list tidak terputus.




Gambar 3.6 Perubahan referensi saat menyisipkan node di tengah
Sumber: Diolah oleh penulis.

```
def insert_after(self, target, data):
    current = self.head
```

```
    while current is not None:
        if current.data == target:
            new_node = Node(data)
            new_node.next = current.next
            current.next = new_node
            return True
        current = current.next

    return False
```

### 3.11.1 Dua Baris Kritis

```
new_node.next = current.next
current.next = new_node
```

Baris pertama membuat node baru menunjuk ke node yang semula berada setelah current. Baris kedua baru mengubah current agar menunjuk ke node baru. Jika urutannya dibalik tanpa menyimpan referensi lama, bagian list setelah current dapat kehilangan jalur akses.

### 3.11.2 Pengujian

```
linked = SinglyLinkedList()
linked.append(10)
linked.append(20)
linked.append(30)

berhasil = linked.insert_after(20, 25)
print('Berhasil:', berhasil)
linked.display()
```

```
Berhasil: True
10 -> 20 -> 25 -> 30 -> None
```

Method mengembalikan True ketika target ditemukan dan penyisipan berhasil. Jika target tidak ada, method mengembalikan False. Pola nilai balik seperti ini memudahkan program pemanggil mengetahui hasil operasi.

## 3.12. Mencari Data pada Linked List

Search pada Singly Linked List pada dasarnya adalah traversal yang berhenti ketika data ditemukan. Karena tidak ada akses indeks langsung, pencarian nilai pada kasus terburuk harus memeriksa seluruh node.

```
def search(self, target):
    current = self.head
    position = 0

    while current is not None:
        if current.data == target:
            return position
        current = current.next
        position += 1

    return -1
```

### 3.12.1 Penjelasan position

Linked list tidak menyimpan indeks seperti array. Variabel position di atas hanyalah penghitung yang dibuat saat traversal agar hasil pencarian mudah dibaca. Nilai -1 digunakan sebagai penanda bahwa target tidak ditemukan.

```
print(linked.search(25))
print(linked.search(100))
```

```
2
-1
```

Kompleksitas worst case search adalah O(n). Best case dapat O(1) apabila target berada pada head, tetapi analisis umum tetap memperhatikan bagaimana waktu tumbuh ketika ukuran data bertambah.

## 3.13. Memperbarui Data Node

Update berdasarkan nilai juga membutuhkan pencarian terlebih dahulu. Setelah node ditemukan, hanya atribut data yang diubah; hubungan next tidak perlu disentuh.

```
def update(self, old_value, new_value):
    current = self.head

    while current is not None:
        if current.data == old_value:
            current.data = new_value
            return True
        current = current.next

    return False
```

```
linked.update(25, 27)
linked.display()
```

```
10 -> 20 -> 27 -> 30 -> None
```

Jika node sudah diketahui secara langsung melalui referensi, update data dapat O(1). Namun jika kita hanya memiliki nilai yang harus dicari dari head, total operasinya O(n) karena proses pencarian.

## 3.14. Menghapus Node Pertama

Menghapus head relatif sederhana. Kita cukup memindahkan head ke node kedua. Node lama tidak lagi menjadi bagian dari list karena tidak ada referensi list yang menuju ke sana.

```
def delete_first(self):
    if self.head is None:
        return False

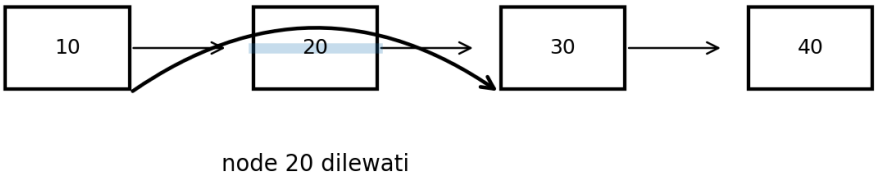
    self.head = self.head.next
    return True
```

Ketika list kosong, self.head bernilai None sehingga tidak ada node yang dapat dihapus. Jika list hanya berisi satu node, self.head.next bernilai None dan setelah operasi head otomatis menjadi None.

Edge case adalah kondisi batas yang mudah terlewat, misalnya list kosong, hanya satu node, target berada di head, target berada di node terakhir, atau target sama sekali tidak ditemukan. Implementasi struktur data harus diuji pada kondisi-kondisi ini.

## 3.15. Menghapus Node Berdasarkan Nilai

Untuk menghapus node yang bukan head, program memerlukan referensi ke node sebelumnya. Alasannya, next milik node sebelumnya harus diubah agar langsung menunjuk ke node setelah target.




Gambar 3.7 Konsep menghapus node dengan melewati node target
Sumber: Diolah oleh penulis.

```
def delete(self, target):
    if self.head is None:
        return False

    if self.head.data == target:
        self.head = self.head.next
        return True

    previous = self.head
    current = self.head.next

    while current is not None:
        if current.data == target:
            previous.next = current.next
            return True

        previous = current
        current = current.next

    return False
```

### 3.15.1 Mengapa Perlu previous dan current?

current digunakan untuk memeriksa node yang mungkin menjadi target, sedangkan previous menyimpan node sebelum current. Ketika target ditemukan, previous.next diarahkan ke current.next. Akibatnya node current dilewati dari rantai utama.

### 3.15.2 Dry Run Delete

| Langkah | previous | current | Kondisi |
|---|---|---|---|
| Awal | 10 | 20 | 20 bukan target 30 |
| Geser | 20 | 30 | 30 adalah target |
| Ubah referensi | 20 | 30 | previous.next = current.next (40) |
| Hasil | 20 menunjuk 40 | 30 dilewati | 10 -> 20 -> 40 -> None |

Pada Python, kita tidak perlu melakukan deallocation manual seperti free pada C. Ketika tidak ada referensi yang membutuhkan object tertentu, pengelolaan memori dilakukan oleh mekanisme Python. Untuk pembelajaran struktur data, fokus kita adalah memastikan hubungan antar-node benar.

## 3.16. Menghitung Jumlah Node

Ukuran linked list dapat dihitung dengan traversal. Versi paling sederhana tidak menyimpan atribut size, sehingga setiap pemanggilan length() harus mengunjungi seluruh node.

```
def length(self):
    count = 0
    current = self.head

    while current is not None:
        count += 1
        current = current.next

    return count
```

```
print('Jumlah node:', linked.length())
```

```
Jumlah node: 4
```

Kompleksitasnya O(n). Alternatif desain adalah menyimpan self.size dan memperbaruinya setiap insert/delete. Dengan cara itu pembacaan ukuran dapat O(1), tetapi programmer harus disiplin agar nilai size selalu konsisten.

## 3.17. Implementasi Singly Linked List Terintegrasi

Berikut satu versi class yang menggabungkan operasi utama. Jalankan cell ini dari awal agar notebook memiliki definisi class terbaru sebelum praktik berikutnya.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def is_empty(self):
        return self.head is None

    def insert_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
```

```
        self.head = new_node

    def append(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node

    def insert_after(self, target, data):
        current = self.head
        while current is not None:
            if current.data == target:
                new_node = Node(data)
                new_node.next = current.next
                current.next = new_node
                return True
            current = current.next
        return False

    def search(self, target):
        current = self.head
        position = 0
        while current is not None:
            if current.data == target:
                return position
            current = current.next
            position += 1
        return -1

    def update(self, old_value, new_value):
        current = self.head
        while current is not None:
            if current.data == old_value:
                current.data = new_value
                return True
            current = current.next
        return False

    def delete(self, target):
        if self.head is None:
            return False
        if self.head.data == target:
            self.head = self.head.next
            return True
        previous = self.head
        current = self.head.next
        while current is not None:
            if current.data == target:
                previous.next = current.next
                return True
            previous = current
            current = current.next
        return False

    def length(self):
        count = 0
        current = self.head
        while current is not None:
            count += 1
            current = current.next
        return count

    def display(self):
        current = self.head
        while current is not None:
```

```
            print(current.data, end=' -> ')
            current = current.next
        print('None')
```

### 3.17.1 Skenario Uji Lengkap

```
data = SinglyLinkedList()

data.append(10)
data.append(20)
data.append(30)
data.insert_beginning(5)
data.insert_after(20, 25)
data.display()

print('Posisi 25:', data.search(25))
print('Jumlah node:', data.length())

data.update(25, 27)
data.delete(20)
data.display()
```

```
5 -> 10 -> 20 -> 25 -> 30 -> None
Posisi 25: 3
Jumlah node: 5
5 -> 10 -> 27 -> 30 -> None
```

Jalankan kode beberapa kali dengan data berbeda. Mahasiswa sebaiknya menggambar kondisi linked list setelah setiap operasi. Jika gambar dan output berbeda, telusuri baris yang mengubah next.

## 3.18. Menelusuri Referensi dengan id()

Python menyediakan fungsi id() yang dapat membantu menunjukkan bahwa variabel berbeda dapat merujuk ke object yang berbeda. Nilainya tidak perlu dihafal dan dapat berbeda setiap kali program dijalankan.

```
a = Node(10)
b = Node(20)
a.next = b

print('id a       :', id(a))
print('id a.next  :', id(a.next))
print('id b       :', id(b))
print(a.next is b)
```

Dua nilai id untuk a.next dan b akan sama pada satu eksekusi karena keduanya merujuk ke object yang sama. Ekspresi a.next is b menghasilkan True. Praktik ini membantu membedakan antara menyalin nilai data dan menyimpan referensi ke object.

## 3.19. Kesalahan Umum pada Linked List

| Kesalahan | Gejala | Penyebab / Perbaikan |
|---|---|---|
| Lupa memindahkan current | Loop tidak pernah selesai | Tambahkan current = current.next pada traversal. |
| Mengubah head pada tempat yang salah | Node awal hilang atau urutan berubah | Gambarkan referensi sebelum assignment. |

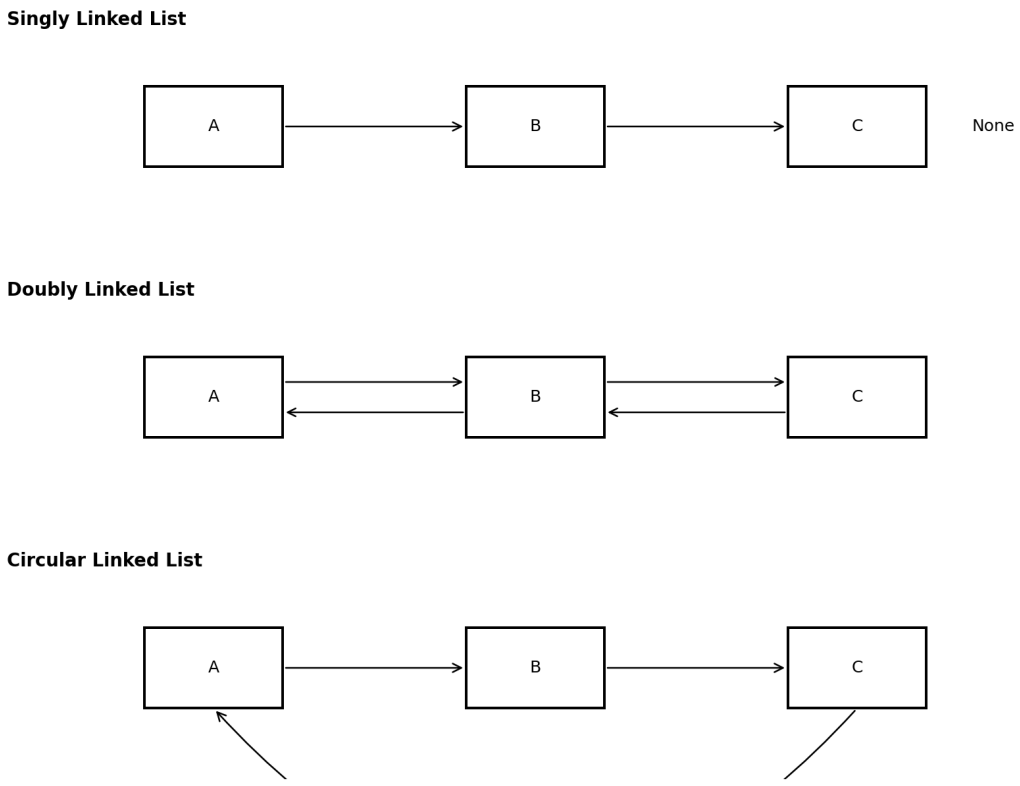
| | | |
|---|---|---|
| Urutan insert salah | Sebagian list terputus | Simpan hubungan lama pada new_node.next terlebih dahulu. |
| Tidak menangani list kosong | AttributeError pada None | Periksa self.head is None. |
| Mengakses current.data saat current None | AttributeError | Pastikan kondisi while mengecek current is not None. |
| Menghapus target tanpa previous | Hubungan antar-node tidak dapat diperbaiki | Simpan previous ketika menghapus selain head. |

### 3.19.1 Contoh Infinite Loop

```
current = self.head
while current is not None:
    print(current.data)
    # current = current.next  <-- terlupa
```

Karena current tidak pernah berubah, kondisi while akan terus benar selama head bukan None. Pada Google Colab, hentikan cell jika loop seperti ini terjadi. Setelah itu periksa apakah variabel pengendali benar-benar diperbarui di dalam perulangan.

## 3.20. Pengantar Doubly dan Circular Linked List




Gambar 3.8 Perbandingan Singly, Doubly, dan Circular Linked List
Sumber: Diolah oleh penulis.

Singly Linked List hanya memiliki arah next. Doubly Linked List menambahkan referensi prev sehingga traversal dapat dilakukan maju dan mundur. Circular Linked List membuat node terakhir kembali menunjuk ke node pertama sehingga tidak berakhir pada None.

Pertemuan ini tidak menuntut implementasi lengkap kedua variasi tersebut, tetapi mahasiswa perlu memahami alasan keberadaannya.

| Jenis | Referensi Utama | Kelebihan Konseptual | Konsekuensi |
|---|---|---|---|
| Singly | next | Struktur paling sederhana, overhead kecil | Navigasi hanya satu arah |
| Doubly | prev dan next | Mudah bergerak dua arah dan menghapus jika node diketahui | Memori dan perubahan referensi lebih banyak |
| Circular | next kembali ke awal | Cocok untuk proses berulang/round-robin | Kondisi berhenti traversal harus lebih hati-hati |

### 3.20.1 Contoh Struktur Node Doubly

```
class DoublyNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

Perbedaan utama terlihat pada atribut prev. Ketika dua node dihubungkan, programmer harus menjaga konsistensi dua arah: node lama menunjuk ke node baru dan node baru menunjuk kembali ke node lama.

## 3.21. Analisis Kompleksitas Operasi

| Operasi | Singly Linked List (head saja) | Alasan |
|---|---|---|
| Cek kosong | O(1) | Hanya memeriksa head. |
| Insert di awal | O(1) | Tidak perlu traversal. |
| Delete di awal | O(1) | Head langsung dipindahkan. |
| Akses posisi ke-i | O(n) | Harus mengikuti next dari head. |
| Search nilai | O(n) | Worst case memeriksa semua node. |
| Append | O(n) | Harus mencari node terakhir. |
| Insert setelah target | O(n) | Target perlu dicari terlebih dahulu. |
| Delete berdasarkan nilai | O(n) | Target dan previous perlu ditemukan. |
| Length tanpa atribut size | O(n) | Semua node dihitung. |

Tabel di atas menggunakan implementasi yang hanya menyimpan head. Struktur dapat dimodifikasi dengan atribut tambahan seperti tail atau size untuk memperbaiki operasi tertentu. Ini menunjukkan bahwa kompleksitas tidak hanya bergantung pada nama struktur data, tetapi juga pada desain implementasinya.

## 3.22. Praktik Mandiri 3.1 — Membangun Linked List dari Nol

Buat notebook baru atau lanjutkan notebook Pertemuan 3. Kerjakan tanpa menyalin class terintegrasi terlebih dahulu.

1. Buat class Node dengan atribut data dan next.
2. Buat tiga node berisi 100, 200, dan 300.
3. Hubungkan node pertama ke kedua, kemudian node kedua ke ketiga.
4. Tampilkan data ketiga node dengan menelusuri dari node pertama tanpa membuat list Python.
5. Gambarkan struktur yang terbentuk pada text cell menggunakan format 100 -> 200 -> 300 -> None.
6. Ubah next node pertama agar langsung menunjuk ke node ketiga. Amati node mana yang sekarang dilewati.
7. Jelaskan dengan kalimat Anda sendiri apa arti sebuah referensi next.

Target praktik: mahasiswa dapat menjelaskan hubungan object dan referensi sebelum menggunakan class SinglyLinkedList yang lebih besar.

## 3.23. Praktik Mandiri 3.2 — Operasi Insert dan Search

1. Buat SinglyLinkedList kosong.
2. Tambahkan data 15, 25, 35, 45 menggunakan append().
3. Tambahkan 5 menggunakan insert_beginning().
4. Sisipkan 30 setelah nilai 25.
5. Tampilkan linked list.
6. Cari nilai 30 dan 99. Catat hasil keduanya.
7. Lakukan dry run pencarian nilai 45 dengan tabel current dan position.
8. Jelaskan mengapa search memiliki kompleksitas O(n).

## 3.24. Praktik Mandiri 3.3 — Delete dan Edge Case

1. Uji delete pada list kosong.
2. Buat list yang hanya berisi satu node lalu hapus node tersebut.
3. Buat list berisi minimal lima node. Hapus head, node tengah, dan node terakhir secara berurutan.
4. Coba hapus nilai yang tidak ada. Pastikan program tidak error.
5. Setelah setiap operasi, tampilkan isi list dan jumlah node.
6. Jelaskan peran previous saat menghapus node yang bukan head.
7. Tambahkan pengujian Anda sendiri untuk menemukan kemungkinan kesalahan implementasi.

## 3.25. Mini Studi Kasus — Daftar Tugas Mahasiswa

Sebuah aplikasi sederhana menyimpan daftar tugas mahasiswa dalam urutan pengerjaan. Setiap tugas mempunyai judul. Pengguna dapat menambah tugas baru, menyisipkan tugas prioritas setelah tugas tertentu, mencari tugas, mengganti judul, dan menghapus tugas yang selesai.

### Data Awal

```
['Membaca Modul', 'Latihan Python', 'Membuat Ringkasan']
```

### Tugas Implementasi

1. Simpan setiap judul sebagai data pada node Singly Linked List.
2. Tambahkan tiga data awal menggunakan append().
3. Tambahkan “Cek Pengumuman” di awal.
4. Sisipkan “Diskusi Kelompok” setelah “Latihan Python”.
5. Cari posisi “Membuat Ringkasan”.
6. Ubah “Membaca Modul” menjadi “Membaca Modul Linked List”.
7. Hapus “Latihan Python”.
8. Tampilkan hasil akhir dan jumlah node.
9. Tuliskan kompleksitas setiap operasi utama yang digunakan.
10. Jelaskan kapan penggunaan linked list pada kasus ini lebih menarik dibanding array/list dan kapan justru kurang praktis.

### Luaran

Luaran berupa satu notebook Google Colab yang dapat dijalankan dari awal sampai akhir tanpa error. Setiap kelompok kode diberi text cell penjelasan. Minimal satu operasi insert dan satu operasi delete harus disertai gambar/dry run perubahan referensi.

## 3.26. Rangkuman

Linked list adalah struktur data linear yang dibentuk oleh node dan referensi antar-node. Pada Singly Linked List, setiap node menyimpan data dan next, sedangkan head menunjuk ke node pertama. Traversal dilakukan dari head dengan mengikuti next hingga mencapai None.

Operasi insert dan delete pada linked list sangat bergantung pada posisi node dan referensi yang tersedia. Insert atau delete di awal dapat dilakukan O(1), sementara search, akses berdasarkan posisi, append dengan head saja, dan delete berdasarkan nilai umumnya O(n). Pemahaman terhadap perubahan referensi merupakan kunci untuk mengimplementasikan linked list dengan benar.

Python digunakan sebagai alat implementasi melalui class Node dan SinglyLinkedList. Praktik pada pertemuan ini menekankan pembacaan kode, dry run, pengujian edge case, serta kemampuan menghubungkan proses program dengan gambar struktur data.

## 3.27. Latihan dan Evaluasi

1. Jelaskan perbedaan cara akses elemen pada array dan linked list.
2. Apa fungsi head pada Singly Linked List?
3. Mengapa node terakhir memiliki next = None?
4. Tuliskan class Node paling sederhana menggunakan Python dan jelaskan setiap atributnya.
5. Lakukan dry run traversal pada struktur 7 -> 14 -> 21 -> 28 -> None.
6. Mengapa insert di awal Singly Linked List dapat O(1)?
7. Mengapa append pada implementasi yang hanya memiliki head dapat O(n)?
8. Jelaskan risiko jika current tidak pernah diperbarui di dalam loop traversal.
9. Tuliskan dua baris perubahan referensi yang diperlukan untuk menyisipkan new_node setelah current.
10. Mengapa delete node selain head membutuhkan informasi tentang node sebelumnya?
11. Apa yang terjadi jika list hanya berisi satu node kemudian delete_first() dipanggil?
12. Buat method contains(target) yang mengembalikan True atau False.
13. Buat method get(index) yang mengembalikan data pada posisi tertentu dan None jika indeks tidak valid.
14. Modifikasi class agar memiliki atribut size yang selalu diperbarui saat insert/delete.
15. Jelaskan perbedaan Singly Linked List, Doubly Linked List, dan Circular Linked List.
16. Sebuah aplikasi sering menambah dan menghapus data di awal tetapi jarang mengakses posisi acak. Bandingkan pertimbangan penggunaan array/list dan linked list.
17. Buat pengujian untuk list kosong, satu node, banyak node, target tidak ditemukan, serta target pada node terakhir.

### Latihan Diskusi

Sebuah sistem pemutar musik memungkinkan pengguna menambahkan lagu, berpindah ke lagu berikutnya, kembali ke lagu sebelumnya, dan mengulang playlist dari awal. Diskusikan jenis linked list yang paling sesuai untuk perilaku tersebut. Jelaskan kebutuhan referensi yang harus tersedia dan konsekuensinya terhadap kompleksitas implementasi.

## DAFTAR PUSTAKA


[1]. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms, 4th ed. MIT Press, 2022.

[2]. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. Data Structures and Algorithms in Python. John Wiley & Sons, 2013.

[3]. Bhargava, A. Y. Grokking Algorithms, Second Edition: An Illustrated Guide for Programmers and Other Curious People. Manning, 2024.

[4]. Miller, B. N., & Ranum, D. L. Problem Solving with Algorithms and Data Structures Using Python. Franklin, Beedle & Associates, 2011.

[5]. Python Software Foundation. The Python Tutorial — Classes. https://docs.python.org/3/tutorial/classes.html.

[6]. Python Software Foundation. Data model — Objects, values and types. https://docs.python.org/3/reference/datamodel.html.

[7]. Google. Google Colaboratory. https://colab.research.google.com/.